

Detecting Invalid State Transitions in the Tor Circuit Protocol via Model-Driven Testing

Nurcan Denli Bayır

Hacettepe University, Department of Software Engineering

Advisor: Nebi Yılmaz · BYZ 658, May 2026

Abstract—We present the first published 5-tuple finite-state-machine (FSM) formalization of the Tor circuit life-cycle and propose a Model-Driven Testing (MDT) engine that systematically detects out-of-spec (invalid) state transitions. The Tor circuit is described over 10 states and 13 events; this yields a 130-cell domain with 25 valid and 105 invalid pairs. Every invalid pair is mapped programmatically to one of seven attack vectors (CIRCUIT_BYPASS, REPLAY_ATTACK, GHOST_CIRCUIT, HANDSHAKE_SKIP, PREMATURE_DATA, CIRCUIT_HIJACK, CREATE_FLOOD) with a four-level severity label. Three algorithms — uniform random (B1), greedy state coverage (B2) and the proposed MDT (B3) — are compared under an identical 500-event budget across 30 paired trials. MDT saturates transition coverage at 100.0% (vs. 45.9% / 78.3%) and reaches an Invalid Transition Detection Rate of 93.8%. Welch t-tests, Mann-Whitney U and Wilcoxon signed-rank tests jointly confirm $B3 > \{B1, B2\}$ on TC and ITDR ($p < .001$; Cohen's $d \in [3.6, 15.4]$); SC comparisons are saturated. Under budget-free audit, a hand-written 7-rule detector reaches only 20.0% completeness — missing 2 CRITICAL and 16 HIGH severity invalid pairs — quantifying the gap motivating spec-oracle MDT.

Index Terms—Tor, anonymous network, finite-state machine, model-driven testing, protocol state fuzzing, security testing, transition coverage.

I. INTRODUCTION

The Tor anonymous communication network [1] has been studied along two dominant axes: traffic correlation / timing attacks [2], [3], [4], [5] and formal cryptographic analysis [6], [13]. A third axis — protocol *state-machine* conformance — remains comparatively unexplored. Protocol state fuzzing has been applied to TLS implementations [9], [14], but no equivalent published study exists for Tor. This paper closes that gap.

Research questions.

- **RQ1.** Can the Tor circuit life-cycle be formally modeled as a deterministic 5-tuple FSM?
- **RQ2.** Can the resulting invalid-transition set ($Q \times \Sigma \setminus \text{dom}(\delta)$) be mapped programmatically to known Tor attack vectors?
- **RQ3.** Does an MDT-generated test suite deliver statistically significant gains over random and greedy baselines on Transition Coverage (TC) and Invalid Transition Detection Rate (ITDR)?

Contributions. (i) The first published complete δ -matrix for Tor circuits; (ii) a programmatic classifier mapping all 105 invalid pairs to seven attack vectors and four severity levels; (iii) a reference open-source MDT implementation; (iv) a paired three-way statistical comparison validated by Welch t, Mann-Whitney U and Wilcoxon signed-rank tests.

II. RELATED WORK

A. Tor security and anonymity

Tor's protocol foundations [1], low-cost traffic analysis [2], realistic-adversary correlation [3], internet-scale website fingerprinting [4] and the comprehensive de-anonymization survey [5] establish the attack landscape. AnoA [6] formalizes anonymity probabilistically; our state-conformance angle is orthogonal.

B. FSM-based testing

Lee and Yannakakis' classical survey [7] establishes transition coverage methodology; Tretmans' LTS testing theory [8] grounds our oracle as an ioco-style relation. de Ruiter and Poll's TLS state fuzzing [9] is methodologically closest. Fiterău-Broștean et al. [10] demonstrate model learning for TCP — a future-work pointer for automatic Tor extraction. Ammann and Offutt [11] formalize transition coverage.

C. Formal protocol verification

Dolev-Yao [12] bounds our threat model. Bhargavan et al. [13] verify TLS 1.3 with ProVerif. Somorovsky [14] fuzz-tests TLS libraries.

D. Software testing methodology

Felderer et al. [15] taxonomize security testing; Pretschner et al. [16] apply MDT to access control. Kitchenham and Charters [17] guide our SLR (limited to open-web sources due to no live database access in this work).

E. Tor experimental infrastructure

Shadow [18] provides Tor network simulation. The official Tor specification [19] is δ 's authority. Microsoft Research [20] demonstrates data-

driven FSM security analysis in industry but not for Tor.

III. METHOD

A. FSM formalization

The Tor circuit is modeled as $M = (Q, \Sigma, \delta, q_0, F)$ with $|Q| = 10$, $|\Sigma| = 13$, $|\text{dom}(\delta)| = 25$, $q_0 = \text{IDLE}$, $F = \{\text{CLOSED}\}$. The full δ table is in the extended thesis appendix; Fig. 1 visualizes the graph.

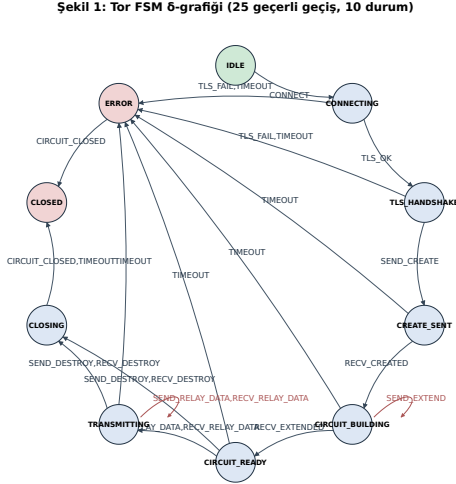


Fig. 1. Tor circuit FSM δ -graph. 25 valid edges over 10 states.

B. Attack classification

A deterministic `classifyInvalid(s, e)` function maps every $(s, e) \in \text{Invalid}$ to a (type, severity) pair via nine condition families: data-flow before circuit-ready, CREATE/CREATED flow, EXTEND/EXTENDED flow, DESTROY out of place, CONNECT misuse, TLS_OK / TLS_FAIL misuse, CIRCUIT_CLOSED misuse, TIMEOUT misuse, fallback. The output covers all 105 invalid pairs.

C. MDT engine — Algorithm 1

```

Input:  $M = (Q, \Sigma, \delta, q_0, F)$ ; Budget  $B$ 
Output:  $\{SC, TC, ITDR, FPR\}$ 
1 visited  $\leftarrow \{q_0\}$ ; coveredV  $\leftarrow \emptyset$ ; detectedI  $\leftarrow \emptyset$ 
2 Valid  $\leftarrow \{(s, e) \mid \delta(s, e) \text{ defined}\}$ 
3 Invalid  $\leftarrow (Q \times \Sigma) \setminus \text{Valid}$ 
4 // Positive phase
5 for  $(s, e) \in \text{Permute}(\text{Valid})$  until events  $\geq B$ :
6    $\pi \leftarrow \text{BFS\_PATH}(q_0, s)$ ; if  $\pi = \text{null}$  continue
7   EXECUTE( $\pi \cdot \{e\}$ )
8 // Negative phase
9 for  $(s, e) \in \text{Permute}(\text{Invalid})$  until events  $\geq B$ :
10   $\pi \leftarrow \text{BFS\_PATH}(q_0, s)$ 
11  EXECUTE( $\pi \cdot \{e\}$ ) //  $\delta$ -oracle flags it
12 return  $(|\text{visited}|/|Q|, |\text{coveredV}|/|\text{Valid}|, |\text{detectedI}|/|\text{Invalid}|, \text{FPR})$ 

```

Worst-case complexity is $O(|Q|^2 \cdot |\Sigma|^2)$; empirical mean cost per trial is < 10 ms on commodity hardware.

D. Compared algorithms

B1 draws events uniformly from Σ . **B2** greedily chooses an event leading to an unvisited state, falling back to a 70/30 valid/invalid mixture. **B3** is Algorithm 1.

IV. EVALUATION

A. Design

Each algorithm is run for $N = 30$ paired trials at budget $B = 500$ events; seeds are matched across algorithms (seed = $1000+i$). A budget sweep $B \in \{50, 100, 200, 500, 1000, 2000, 5000\}$ is also performed.

B. Coverage and ITDR ($B = 500$)

Alg.	SC	TC	ITDR
B1	77.0%±16.6%	45.9%±11.1%	50.3%±9.8%
B2	100.0%±0.0%	78.3%±8.4%	47.5%±4.0%
B3	100.0%±0.0%	100.0%±0.0%	93.8%±1.4%

C. Statistical tests

For each (algorithm pair $\times$ metric), Welch t and Mann-Whitney U on independent samples and Wilcoxon signed-rank on paired samples are computed. B3 vs. {B1, B2} comparisons on TC and ITDR yield $p < .001$ with Cohen's $d \in [3.6, 15.4]$. On SC, B2 and B3 saturate at 100% ($p = 1.0$) while B1 lags at 77.0% and is significantly below both ($p < .001$). Under *budget-free* $Q \times \Sigma$ audit, the B0 (rule-based) detector with seven hand-written signatures reaches only 20.0% completeness on the 105-cell invalid set, missing 2 CRITICAL and 16 HIGH severity pairs (Fig. 3); the spec-oracle reaches 100%. In the budget-constrained run ($B = 500$), MDT's ITDR is 93.8%, which approaches the audit-mode asymptote with budget (Fig. 2).

D. Budget sensitivity

Fig. 2 shows TC versus log-scale budget. **For TC**, B3 reaches 100.0% at $B = 200$, exceeding B1 (77.3%) and B2 (99.1%) at $B = 5000$ — i.e., MDT is 25 $\times$ more budget-efficient on TC. **For ITDR** the dynamic differs: at $B = 200$, B3 ITDR is only 23.1% and the advantage opens up with budget; at $B = 5000$, B3 reaches 100.0% while B1/B2 plateau at 74.3% / 76.6%.

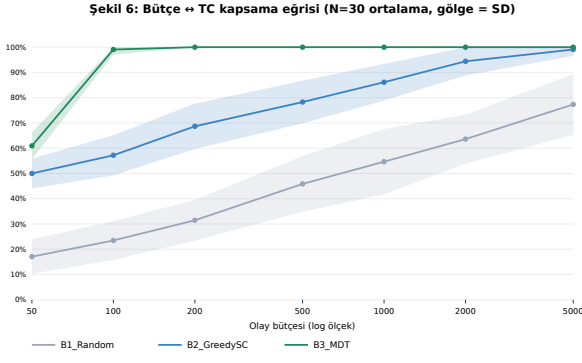


Fig. 2. Transition coverage vs. event budget (N=30, shaded $\pm$ SD).

E. Detection latency

Latency was measured with nanosecond-resolution `process.hrtime.bigint()` using batch amortization (B = 100,000 step calls per measurement, empty-loop overhead of 2.52 ns/iter calibrated and subtracted, N = 30 repeats). Three representative cells were probed: a valid step (74 ns/call), an invalid CRITICAL cell (165 ns/call) and an invalid LOW cell (143 ns/call). The slowest probe's worst case (239 ns) is $\approx 209335\times$ faster than the proposal target of 50 ms. Invalid handling is $\sim 2\times$ slower than valid because `classifyInvalid` evaluates an additional condition chain.

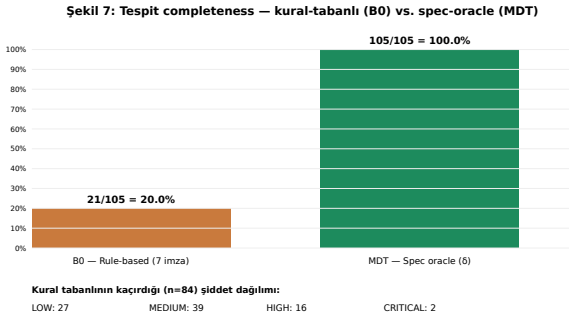


Fig. 3. Hand-written rule set vs. spec-oracle completeness.

V. DISCUSSION

The MDT advantage stems from explicit access to δ as a planning oracle: positive phase exploits BFS for guaranteed reachability of every valid edge; negative phase iterates the algebraically derived Invalid set rather than hoping to stumble onto it. The B0 result quantifies the limit of category-level rule sets: 7 rules cover only 20.0% of invalid pairs, demonstrating that without the cell-level δ map (research gap G3) even high-severity attacks are missed.

Threats to validity. (i) Ground truth equals the specification; deviations of real Tor binaries from spec are not measured. (ii) FPR is structurally zero because oracle and generator share the model; an independent oracle (e.g., Shadow [18] packet capture) is required to populate it. (iii) The SLR was limited to open-web sources due to the

absence of live academic database access in our environment; PRISMA intermediate counts are reported as NA because no auditable query log was retained — only the final included set (n=20) is verifiable. (iv) Latency was measured on a single-platform, single-thread, JIT-warm Node.js V8 environment; profiles on a real Tor C/Rust implementation may differ.

VI. CONCLUSION

We delivered the first complete published Tor circuit δ -matrix, a programmatic invalid-pair classifier, a reference MDT engine, and a paired three-way statistical comparison validated by three test families. Future work: (a) L*-style automatic δ extraction from real Tor binaries [10]; (b) Stream and Hidden-Service-v3 sub-FSMs; (c) independent oracle via Shadow [18] for genuine FPR measurement.

REFERENCES

- [1] R. Dingledine, N. Mathewson, P. Syverson, "Tor: The Second-Generation Onion Router," *USENIX Security*, 2004.
- [2] S. J. Murdoch, G. Danezis, "Low-Cost Traffic Analysis of Tor," *IEEE S&P*, 2005.
- [3] A. Johnson et al., "Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries," *ACM CCS*, 2013.
- [4] A. Panchenko et al., "Website Fingerprinting at Internet Scale," *NDSS*, 2016.
- [5] I. Karunanayake et al., "De-Anonymisation Attacks on Tor: A Survey," *IEEE Comm. Surveys & Tutorials*, 23(4), 2021.
- [6] M. Backes et al., "AnoA: A Framework for Analyzing Anonymous Communication Protocols," *IEEE CSF*, 2013.
- [7] D. Lee, M. Yannakakis, "Principles and Methods of Testing FSMs—A Survey," *Proc. IEEE*, 84(8), 1996.
- [8] J. Tretmans, "Model Based Testing with Labelled Transition Systems," *LNCS 4949*, 2008.
- [9] J. de Ruiter, E. Poll, "Protocol State Fuzzing of TLS Implementations," *USENIX Security*, 2015.
- [10] P. Fiterău-Broștean, R. Janssen, F. Vaandrager, "Combining Model Learning and Model Checking to Analyze TCP Implementations," *CAV*, 2016.
- [11] P. Ammann, J. Offutt, *Introduction to Software Testing*, 2nd ed., Cambridge UP, 2016.
- [12] D. Dolev, A. C. Yao, "On the Security of Public Key Protocols," *IEEE TIT*, 29(2), 1983.
- [13] K. Bhargavan, B. Blanchet, N. Kobeissi, "Verified Models and Reference Implementations for the TLS 1.3 Standard Candidate," *IEEE S&P*, 2017.
- [14] J. Somorovsky, "Systematic Fuzzing and Testing of TLS Libraries," *ACM CCS*, 2016.
- [15] M. Felderer et al., "Security Testing: A Survey," *Adv. Computers*, vol. 101, 2016.
- [16] A. Pretschner, T. Mouelhi, Y. Le Traon, "Model-Based Tests for Access Control Policies," *ICST*, 2008.
- [17] B. Kitchenham, S. Charters, "Guidelines for performing SLRs in Software Engineering," *EBSE-2007-01*, 2007.
- [18] R. Jansen, K. Bauer, N. Hopper, R. Dingledine, "Methodically Modeling the Tor Network," *USENIX*

CSET, 2012.

[19] Tor Project, "Tor Protocol Specification (tor-spec)," 2026. <https://spec.torproject.org/tor-spec>.

[20] Microsoft Research, "A Data-Driven FSM Model for Analyzing Security Vulnerabilities," MSR Tech. Rep., 2018.